Fundamental Programming Structures in JAVA

ABSTRACT CLASS

Objectives

- To introduce the Abstraction.
- To design and use abstract classes.
- To define abstract methods.

Abstraction in Real Life

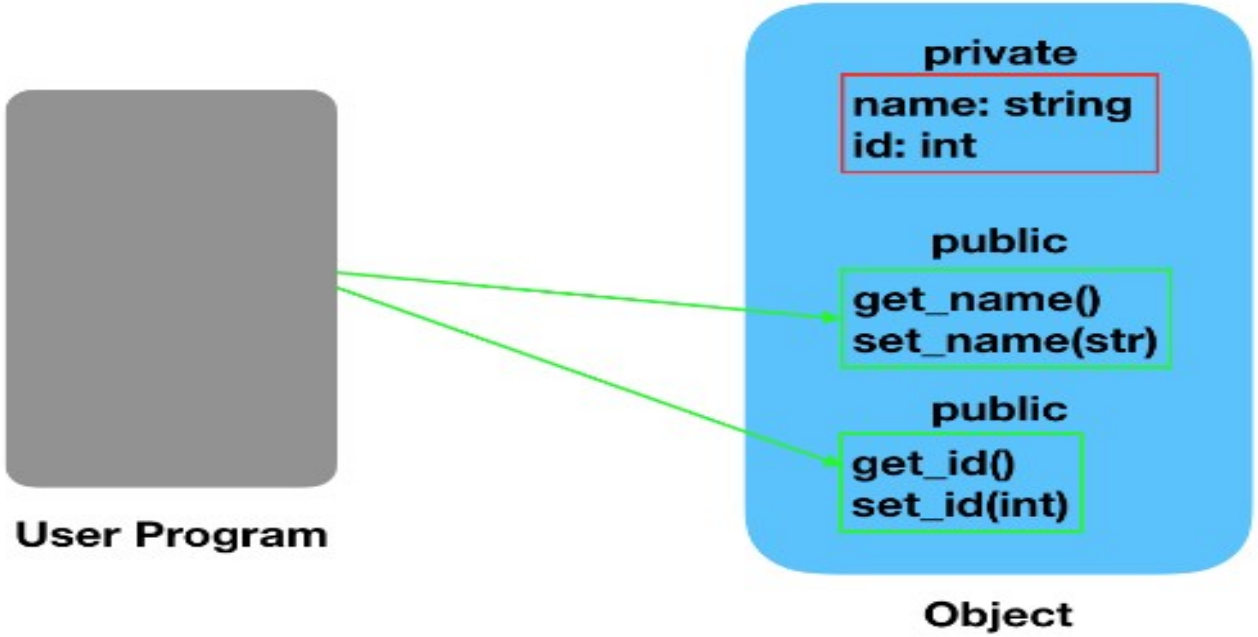
- A man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car.
- But the man does not know about the inner mechanism of the car or the implementation of accelerator, brakes.. etc in the car.

Abstraction in OOP

- Objects are the building blocks of Object-Oriented Programming.
- An object contains some properties and methods.
- In OOP can hide properties and methods from the outer world through access modifiers.
- OOP can provide access only for required methods and properties to the other programs.
- This is the general procedure to implement abstraction in OOPS.
- There are two types of Abstraction: **Data Abstraction** and **Process (Method) Abstraction**.

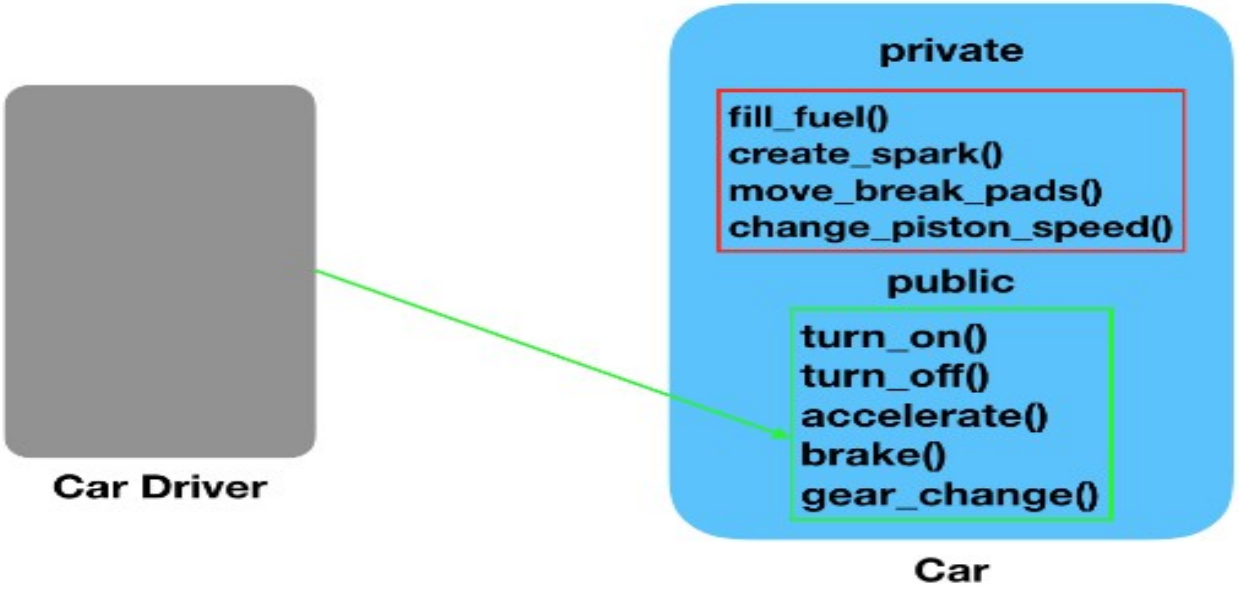
Data Abstraction

- When the object data is not visible to the outer world, it creates data abstraction. If needed, access to the Objects' data is provided through some methods (such as setters and getters).



Process Abstraction

- We don't need to provide details about all the functions of an object. When we hide the internal implementation of the different functions involved in a user operation, it creates process abstraction.



Abstraction in Java

- Abstraction lets you focus on what the object does instead of how it does it.
- Abstraction in Java is implemented through **abstract classes** and **interfaces**. They are used to create a base implementation or contract for the actual implementation classes.

Abstract Classes

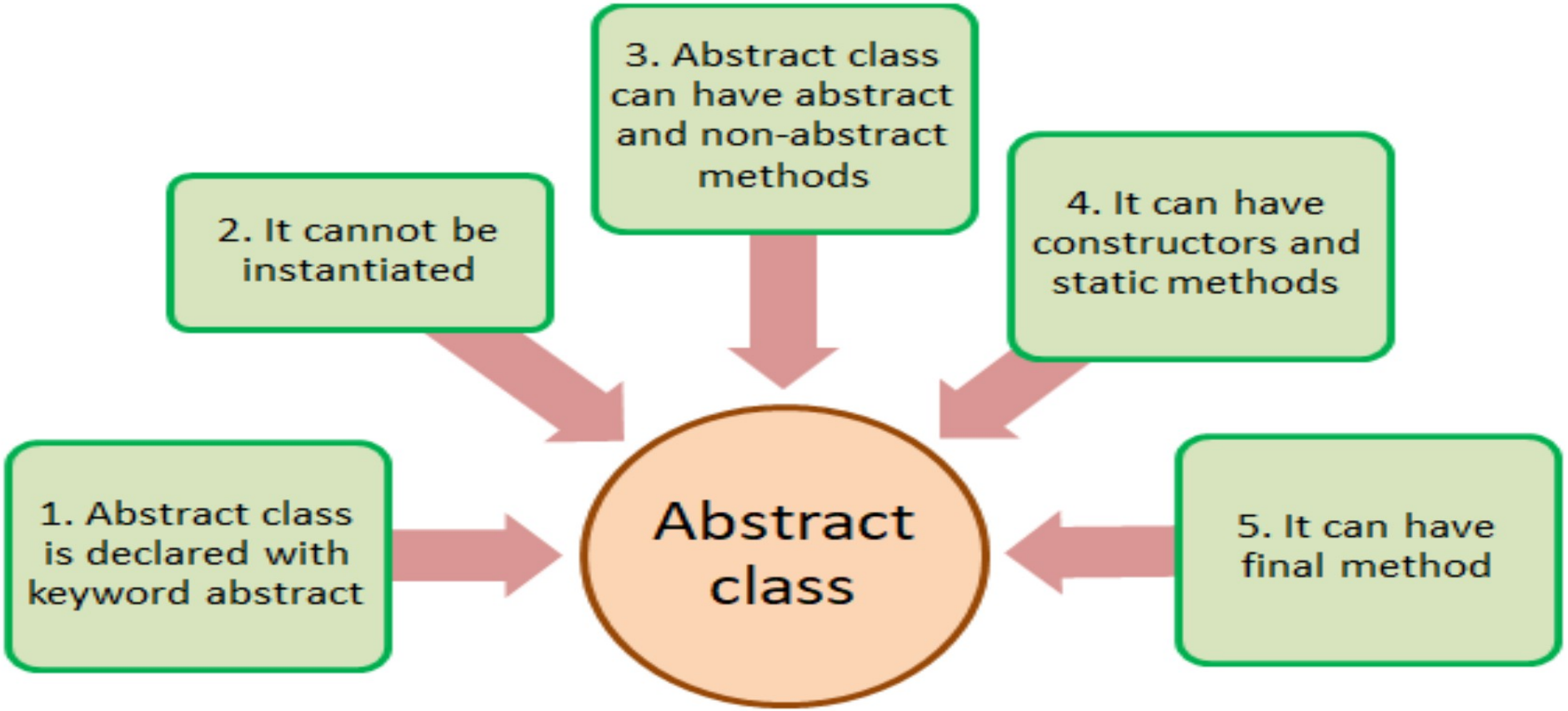
```
public abstract class ClassName {…….}
```

- An abstract class **cannot be instantiated** (cannot be used to create object), but other classes are derived (inherited) from it.
- An ***Abstract class*** serves as a superclass for other classes.
- The abstract class represents the generic or abstract form of all the classes that are derived from it.
- A class becomes abstract when you place the abstract key word in the class definition.

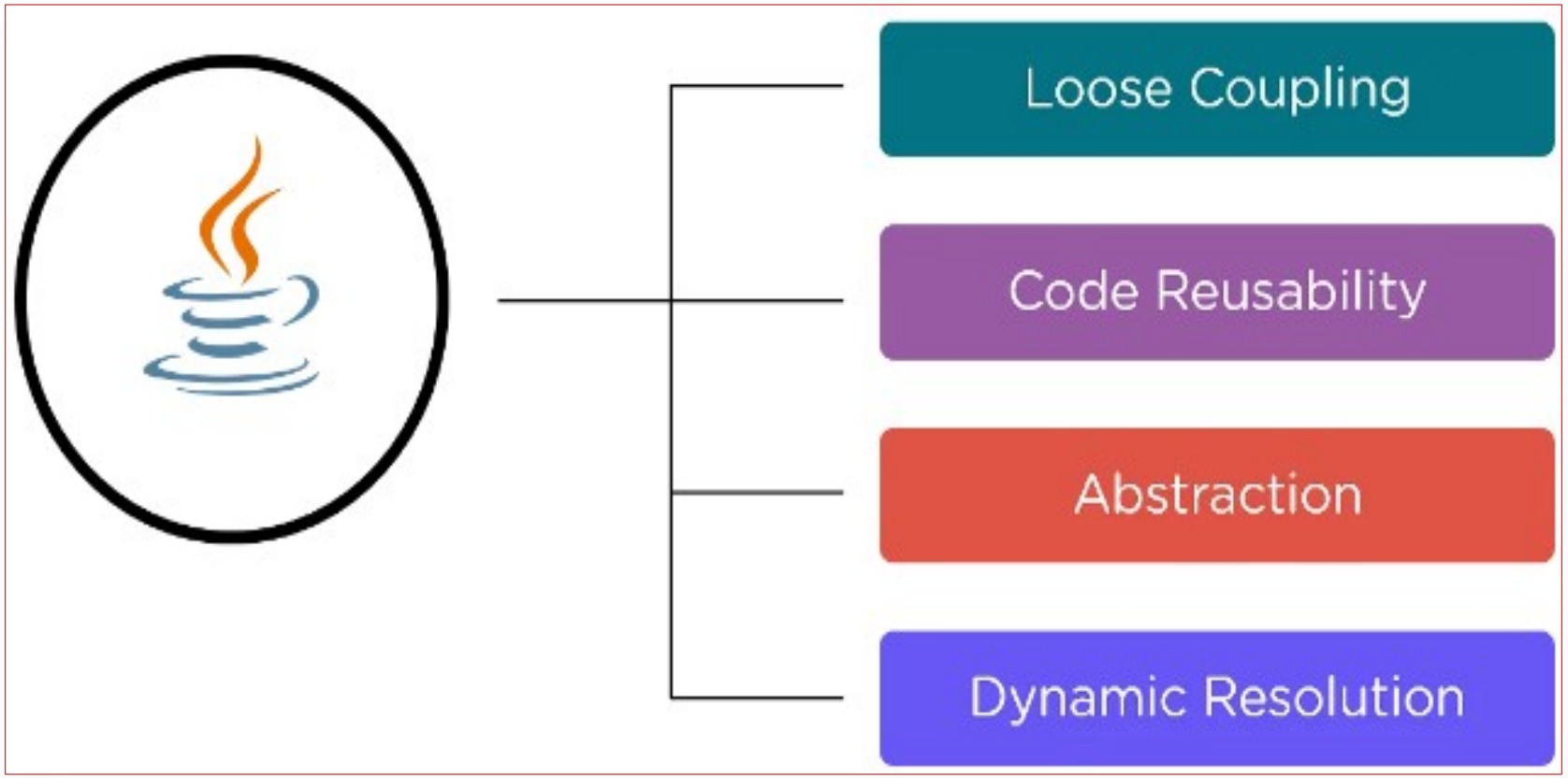
Abstract Classes

- A class that contains abstract methods must be abstract.
- An abstract class can have abstract and no abstract methods.
- It is possible to define an abstract class that contains no abstract methods.
- It can have constructors and static methods.
- It can have final method.

Abstract Classes



Abstract Classes Features



Abstract Classes Features

- **Loose Coupling**: Data abstraction in Java enables loose coupling, by reducing the dependencies at an exponential level.
- **Code Reusability**: Using an abstract class in the code saves time. We can call the abstract method wherever the method is necessary. Abstract class avoids the process of writing the same code again.
- **Abstraction**: Data abstraction in Java helps the developers hide the code complications from the end-user by reducing the project's complete characteristics to only the necessary components.
- **Dynamic Resolution**: Using the support of dynamic method resolution, developers can solve multiple problems with the help of one abstract method.

Abstract Methods

- An abstract method has no body and must be overridden in a subclass.
- An abstract method cannot be contained in a no abstract (concrete) class.
- An abstract method is a method that appears in a super class, but expects to be overridden in a subclass.
- An abstract method has only a header and no body.

Abstract Methods

- Notice that the key word **abstract** appears in the header, and that the header ends with a semicolon.

```
public abstract void setValue(int value);
```

- Any class that contains an abstract method is automatically abstract.
- If a subclass fails to override an abstract method, a compiler error will result.
- Abstract methods are used to ensure that a subclass implements the method.
- A subclass can be abstract even if its superclass is concrete. For example, the Object class is concrete, but its subclasses, such as **Vehicle** , may be abstract.

Abstract class and abstract methods

A class Template

Class Name
List of Attributes …..
$Method_1$ name (): return data type or void $Method_2$ name (): return data type or void $Method_3$ name (): return data type or void . . $Method_n$ name (): return data type or void ;

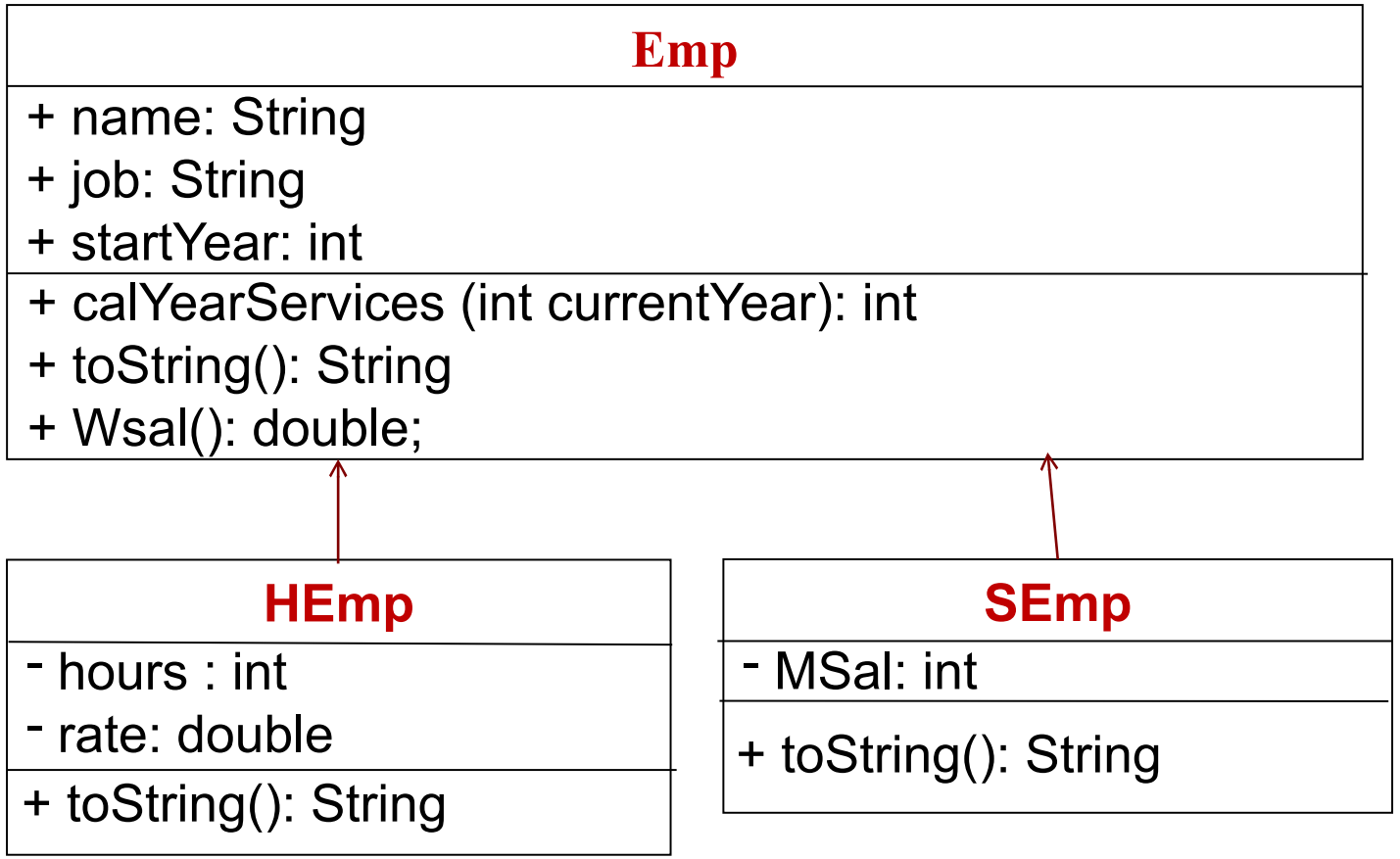
- The special thing about the abstract class is that it **could** have abstract methods; You can distinguish this by having a semicolon at the end of the method definition.

Note: an abstract method cannot be written in a non-abstract class. While an abstract class can have a non-abstract method.

Abstract Class and Abstract Methods Example

- **Example:** A company has two types of employees. One type gets a monthly salary (**SEmp**), and the other gets an hourly-based salary (**HEmp**). The company keeps the following information for each employee (Name, job, and start year) Also, keep tracking the (salary for a salary-based employee, the number of hours worked, and the hourly rate for an hourly employee). They would like to have an automated way to calculate their salary and how many years they have been in service. Moreover, they would like to store all employees in one place and calculate the total amount of money that the company will pay them based on their reports.

Abstract Class and Abstract Methods Example (Design Phase)



Abstract Class and Abstract Methods Example

- When can we define a method as an abstract method?
  Abstract method is a method that is applicable to all subclasses but **with different behaviors** and this method has no meaning (i.e. no body) in the superclass.
- Emp class includes an abstract method called **Wsal()**, which **necessitates** definition within an abstract class.
- **Wsal()** is an abstract method that has only a header **without** body.
- The benefit of defining an abstract method is **to force all the subclasses implementing this method.**
- All subclasses (SEmp and HEmp) **must** override all abstract methods. Otherwise, compile-time errors will appear in all the subclasses that inherit from the abstract class.

Abstract Class and Abstract Methods Example (implementation phase)

```
abstract class Emp {
    protected String name;
    protected String job;
    protected int startYear;

    public Emp(){ name="unknown"; job="unknown"; startYear=0;   }
    public Emp(String name){ this(); this.name = name;}
    public Emp(String name, String job) { this(name);this.job=job;}
    public Emp(String name, String job, int sy){this(name,job); startYear=sy;}

    public int calYearServices(int currentYear) {
        return currentYear - startYear;    }

    public String toString() {
        return "Name:" + name + " job:" + job + " Starting-at:" + startYear; }

    public abstract double Wsal();
```

Wsal() is applicable to SEmp and Hemp and has no behaviour (i.e. no body) in the Emp class, simply because it has a different behavior (implementation) in each of the **subclasses**. This will simplify the code and bypass compilation-time errors as shown.

Abstract Class and Abstract Methods Example (implementation phase)

```java
class SEmp extends Emp {
    private double MSal;

    public SEmp(String name, String job, int sy, double salary)
            {super(name,job,sy); this.MSal=salary;  }
    public double Wsal() {
        return MSal * 12 / 52;
    }

    public String toString() {
        return super.toString() + "\n" + " His salary = " + MSal;
    }
}
```

The **SEmp class** provides a **customized implementation** of the abstract method **Wsal()**. Therefore, to incorporate **SEmp**'s **Wsal()** method functionality, which essentially (computes the employee's monthly salary), you must **override** the method and appropriately **code its functionality within its body to fulfill the purpose of this method.**

Abstract Class and Abstract Methods Example (implementation phase)

```
class HEmp extends Emp {
    private int hours;
    private double rate;
    public HEmp(String name, String job,
            int sy, int hours, double rate)
    {super(name,job,sy);
     this.hours=hours; this.rate=rate;  }

    public double Wsal() {
        return hours * rate;
    }

    public String toString() {
        return super.toString() + "\nNo of Hours:" + hours + " rate=" + rate;
    }
}
```

Also, the **HEmp class** provides a **customized implementation** of the abstract method **Wsal()**. Therefore, to incorporate **HEmp**'s **Wsal()** method functionality, which essentially (computes the employee's hourly salary), you must **override** the method and appropriately **code its functionality within its body to fulfill the purpose of this method.**

Abstract Class and Abstract Methods Example (implementation phase)

Emp.java | SEmp.java | HEmp.java | test.java

```
1   public class test {
2
3       public static void main(String[] args) {
4           Emp a;  //allowed
5           a = new Emp(); // Not Allowed
6
```

Note that the Emp class cannot be directly instantiated (i.e., created as an object) as demonstrated in line 5. However, a pointer can be defined as depicted in line 4. Please refer to the accompanying figure to review the error.

```
public class test {

    public static
        Emp a;  //
        a = new Emp(); // Not Allowed
```

Emp is abstract; cannot be instantiated

(Alt-Enter shows hints)

Exercise (part 1)

- Write a class named User with the following specifications:
  - Two protected instance variables:
    - **userName** of type String.
    - **email** of type String.
  - One private instance variable:
    - **password** of type String, which must consist of a minimum of 8 characters.
  - One constructor with full parameters.
  - An abstract public method **logIn(enteredUName, enteredPass)** that returns a String value.
  - Mutator and accessor methods (getter/setter) for **password**.
  - toString method that returns the values of all the data fields in the following format:
    - User: ----------, Email: ---------Password: ---------.

Exercise (part 2)

- Write a subclass named **PremiumUser** that inherits from the User class, with the following specifications:
  - Two default instance variables:
    - **subscriptionType** of type String.
    - **subscriptionFee** of type double.
  - One constructor with full parameters.
  - An abstract public method **logIn(enteredUName, enteredPass)** to return “you are successfully logged in” if the entered password and entered username match to userName and password, otherwise return “try again”.
  - Override toString method that returns the values of all the data fields.

Exercise (part 3)

- Write a Java class named **PremiumUserDemo** that creates an object of type **PremiumUser**. Invoke **logIn(enteredUName, enteredPass)** “Enter any values you want to test ” then **print the return value and print user details on the screen**.

References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes :edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual